



DIRECCIÓN GENERAL DE EDUCACIÓN
TECNOLÓGICA INDUSTRIAL Y DE SERVICIOS



MANUAL TÉCNICO DE GIT

Enfoque en el Ciclo de Vida del Control de
Versiones

Introducción

Git es un sistema de control de versiones que permite gestionar la evolución de un proyecto mediante registros llamados commits.

En este modo, Git funciona como un sistema de control de versiones que te permite:

- Registrar cada cambio que haces en tus archivos.
- Volver atrás en el tiempo si algo salió mal.
- Crear ramas para experimentar sin romper tu proyecto.
- Comparar versiones y entender exactamente qué cambió y cuándo.
- Llevar un historial ordenado de tu trabajo como si fuera un diario técnico.

Su funcionamiento sigue un ciclo de vida compuesto por tres áreas principales:

1. Directorio de trabajo (Working Directory)
2. Área de preparación (Staging Area)
3. Repositorio local (Local Repository)

Entre sus comandos principales se encuentran: `git clone`, `git init`, `git status`, `git add` y `git commit` que permiten recorrer ese ciclo de manera controlada.

1.Ciclo de Vida de los Archivos en Git

Cada archivo dentro del proyecto pasa por varios estados:

1. Untracked (No rastreado)

-El archivo existe en la carpeta, pero Git no lo controla.

2. Tracked (Rastreado)

-Git ya lo reconoce; puede estar:

- Modificado (modified)

- Sin cambios (unmodified)
- Preparado (staged)

3. Staging Area

-Es la zona intermedia donde el usuario selecciona qué cambios se enviarán al próximo commit. Además de que permite tener control sobre qué versiones se registrarán.

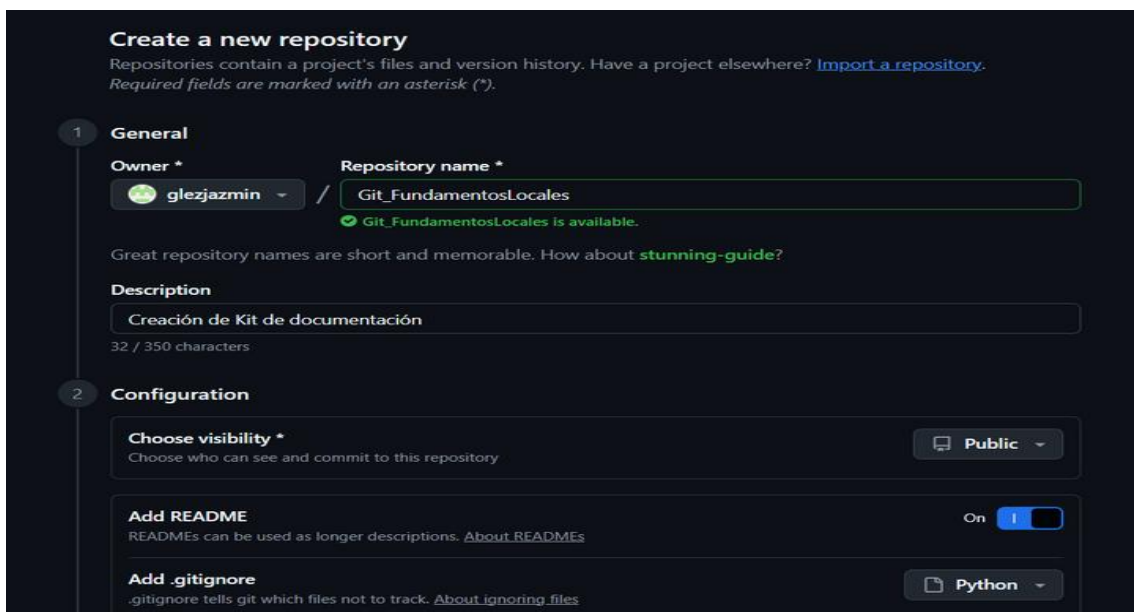
4. Repository / Commit

-Lugar donde Git almacena permanentemente los cambios confirmados

2.Creación del repositorio:

Con estos conocimientos teóricos vamos a empezar a programar nuestro proyecto y crear nuestro repositorio local.

Lo primero que se hace es crear un repositorio en GitHub, el cual nos servirá como un puente para compartir la información de VS Code a PythonAnywhere.



The screenshot shows the GitHub 'Create a new repository' interface. It is divided into two main sections: 'General' and 'Configuration'.

General Section:

- Owner *:** A dropdown menu showing 'glezjazmin'.
- Repository name *:** A text input field containing 'Git_FundamentosLocales'. Below the field, a green checkmark indicates 'Git_FundamentosLocales is available'.
- Description:** A text input field containing 'Creación de Kit de documentación'. Below the field, it says '32 / 350 characters'.

Configuration Section:

- Choose visibility *:** A dropdown menu showing 'Public'.
- Add README:** A toggle switch labeled 'On' is turned on.
- Add .gitignore:** A dropdown menu showing 'Python'.

Para poder ejecutar el repositorio en VS Code, se tiene que copiar el link en el apartado “Code”.



Ya que copiemos el link del repositorio tenemos que clonarlo en PowerShell para poder ejecutarlo en VS Code con el siguiente comando:

Comando: git clone

Función: Su función principal es crear una copia local de un repositorio remoto y dejar los archivos en Working Directory.

Uso:

```
PS C:\Users\gonza\django_AJGH> git clone https://github.com/glezjazmin/Git_FundamentosLocales.git
Cloning into 'Git_FundamentosLocales'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
```

Efecto en el ciclo de vida:

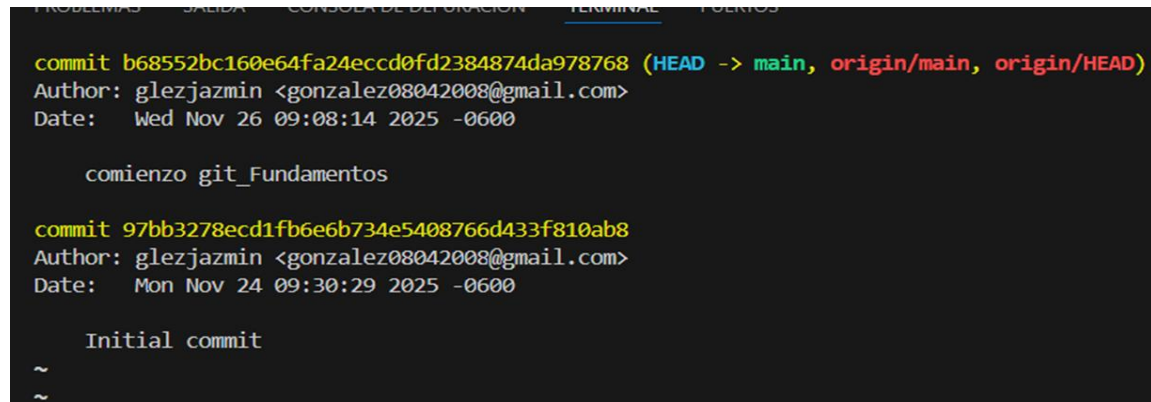
- Esto crea la carpeta proyecto/ con el contenido y el repositorio .git dentro.

El siguiente comando que se utiliza es:

Comando: git init

Función: Este se encarga de crear un repositorio Git en la carpeta actual.

Uso:



```
commit b68552bc160e64fa24eccd0fd2384874da978768 (HEAD -> main, origin/main, origin/HEAD)
Author: glezjazmin <gonzalez08042008@gmail.com>
Date:   Wed Nov 26 09:08:14 2025 -0600

    comienzo git_Fundamentos

commit 97bb3278ecd1fb6e6b734e5408766d433f810ab8
Author: glezjazmin <gonzalez08042008@gmail.com>
Date:   Mon Nov 24 09:30:29 2025 -0600

    Initial commit

~
~
```

Efecto en el ciclo de vida:

- A partir de este momento, Git comenzará a rastrear el proyecto mediante el directorio oculto .git.

3.Verificación del Estado del Proyecto

Para poder verificar el estado del proyecto y visualizar los cambios hechos se utiliza:

Comando: git status

Función: Se encarga de mostrar el estado de los archivos en cada fase del ciclo de vida:

- Archivos no rastreados
- Archivo modificados
- Archivos en Staging Area
- Rama activa

Uso:

```
(venv) PS C:\Users\gonza\django_AJGH\Git_FundamentosLocales> git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        git_fundamentos/
        manage.py
        nucleo/
        requirements.txt

nothing added to commit but untracked files present (use "git add" to track)
(venv) PS C:\Users\gonza\django_AJGH\Git_FundamentosLocales>
```

Efecto en el ciclo de vida:

- Ayuda a identificar qué archivos necesitan ser agregados o confirmados.

4.Preparación de Cambios

En esta parte del proyecto nos ubicamos en la staging área poder realizar cambio y que estos se guarden.

¿Qué es la Staging Area?

- Es el espacio donde Git guarda temporalmente los archivos seleccionados para incluirse en el próximo commit.

Permite:

- Revisar qué se registrará antes de guardarlo
- Realizar commits limpios y organizados
- Evitar registrar cambios incompletos

Unos de sus principales comandos es el:

Comando: `git add .`

Función: Se usa principalmente para agregar todos los cambios hechos.

Uso común:

```
nothing added to commit but untracked files present (use "git add" to track)
(venv) PS C:\Users\gonza\django_AJGH\Git_FundamentosLocales> git add .
```

Efecto en el ciclo de vida:

- Los cambios pasan de Working Directory → Staging Area.

5.Registro Permanente de Cambios

Para finalizar y poder subir todos los cambios realizados se utiliza:

Comando: git commit

Función: Es crear un punto de guardado en el repositorio local con todos los archivos que están en la Staging Area.

Uso:

```
(venv) PS C:\Users\gonza\django_AJGH\Git_FundamentosLocales> git commit -m "comienzo git_Fundamentos"
[main b68552b] comienzo git_Fundamentos
16 files changed, 277 insertions(+)
create mode 100644 .gitignore
create mode 100644 git_fundamentos/__init__.py
create mode 100644 git_fundamentos/asgi.py
create mode 100644 git_fundamentos/settings.py
create mode 100644 git_fundamentos/urls.py
create mode 100644 git_fundamentos/wsgi.py
create mode 100644 manage.py
create mode 100644 nucleo/__init__.py
create mode 100644 nucleo/admin.py
create mode 100644 nucleo/apps.py
create mode 100644 nucleo/migrations/__init__.py
create mode 100644 nucleo/models.py
create mode 100644 nucleo/tests.py
create mode 100644 nucleo/urls.py
create mode 100644 nucleo/views.py
create mode 100644 requirements.txt
(venv) PS C:\Users\gonza\django_AJGH\Git_FundamentosLocales>
```

Efecto en el ciclo de vida:

- Los archivos pasan de Staging Area → Repository, quedando registrados con un identificador único.

Resumen del Ciclo de Vida con Comandos

Para poder entender un poco el ciclo de vida de los comandos y su importancia, se puede visualizar de la siguiente manera:

Fase	Área	Acción	Comando
1	Repository remoto / Working	→ Clona un repositorio	→ <code>git clone</code>
2	Working Directory	→ Crea el repositorio	→ <code>git init</code>
3	Working / Staging	→ Revisa el estado del repositorio	→ <code>git status</code>
4	Staging Area	→ Prepara los cambios	→ <code>git add</code>
5	Repository	→ Guarda los cambios del repositorio	→ <code>git commit</code>

Conclusión

Comprender el ciclo de vida de Git nos asegura tener un control, orden y trazabilidad en cualquier proyecto.

En términos operativos, nos permite gestionar los cambios de forma ordenada, reducir riesgos operativos, optimizar la productividad individual y mantener un historial claro y auditable del proyecto.

Al dominar los comandos fundamentales del Git como `git clone`, `git init`, `git status`, `git add` y `git commit`, el desarrollador comprende el flujo esencial para versionar de manera profesional y enfrentar proyectos complejos que se pongan en su camino de manera eficiente.